UNIVERSIDADE VILA VELHA
Ciência da Computação

Rhayssa dos Santos Rosa da Silva

PROJETO E ANÁLISE DE ALGORITMOS: Explorando a Intratabilidade e a Força Bruta no Problema do Caixeiro Viajante (TSP)

Vila Velha - ES
05 de novembro de 2026

Sumário

1 INTRODUÇÃO 2
2 DESENVOLVIMENTO 2
2.1 Implementação e Coleta de Dados 2
2.2 Validação da Redução do Espaço de Busca 2
2.3 Comportamento do Custo de Avaliação 3
2.4 Resultados Experimentais e Complexidade Total 3
2.5 Comparação entre a Curva Teórica e a Empírica 5
2.6 O Limiar da Intratabilidade 6
2.7 Reflexão sobre P, NP e Heurísticas 6
3 CONCLUSÃO 7

REFERÊNCIAS 8

1 INTRODUÇÃO

O Problema do Caixeiro Viajante (Travelling Salesman Problem – TSP) consiste em encontrar a rota de menor custo que passe por todas as cidades exatamente uma vez e retorne à cidade inicial.

O objetivo deste trabalho é implementar uma solução por força bruta para o problema, utilizando backtracking para gerar e avaliar todas as rotas possíveis. A partir dos resultados obtidos, serão analisados o número de operações realizadas, o tempo de execução e o crescimento da complexidade, comparando os dados experimentais com a previsão teórica apresentada na unidade (Universidade Vila Velha, 2026).

2 DESENVOLVIMENTO

2.1 Implementação e Coleta de Dados

O grafo foi representado por uma matriz de custos simétrica, na qual cada posição indica o custo de deslocamento entre duas cidades. Os valores foram gerados aleatoriamente, utilizando uma semente fixa para garantir que os experimentos pudessem ser reproduzidos.

A implementação utiliza backtracking recursivo para gerar todas as permutações das cidades, mantendo a cidade de índice zero como ponto inicial. Quando uma rota completa é formada, seu custo é calculado pela soma das n arestas, incluindo o retorno à cidade de origem. O algoritmo compara os custos encontrados e mantém a rota de menor valor.

Para acompanhar as execuções, foram utilizados dois contadores: `rotas_avaliadas`, que registra a quantidade de rotas completas examinadas, e `avaliacoes_arestas`, que registra quantas avaliações de custos de arestas foram realizadas.

O tempo de execução foi medido com a função `perf_counter()`, sem considerar a geração da matriz de custos.

Os testes foram realizados para diferentes quantidades de cidades, começando em $n = 3$ e avançando até $n = 14$. Também foi iniciada uma execução para $n = 15$, mas ela não foi concluída após mais de 19 horas.

2.2 Validação da Redução do Espaço de Busca

Conforme apresentado na Seção 4.1.2 da unidade, fixar a cidade inicial evita gerar diferentes representações de um mesmo ciclo, que mudam apenas o ponto de partida.

Com a cidade inicial fixa, restam $n - 1$ cidades para serem organizadas. Portanto, a quantidade de rotas geradas é:

$$R(n) = (n-1)!$$

Caso a cidade inicial não fosse fixada, todas as n cidades seriam permutadas, resultando em:

$$R(n) = n!$$

Assim, a quantidade de rotas seria n vezes maior, pois:

$$\frac{n!}{(n-1)!} = n$$

Na implementação, essa redução é feita retirando a cidade inicial da lista utilizada na geração das permutações. A quantidade de rotas contabilizadas pelo programa é comparada com $(n-1)!$, confirmando que o código realiza a redução esperada.

2.3 Comportamento do Custo de Avaliação

Cada rota completa possui n arestas, pois percorre todas as cidades e retorna à cidade inicial.

Na implementação, a função responsável pelo cálculo do custo percorre essas arestas e soma seus respectivos valores. Como cada avaliação possui custo constante, o número de operações cresce proporcionalmente à quantidade de cidades.

Portanto, a complexidade para avaliar uma única rota é:

$$T_{\text{rota}}(n) = \Theta(n)$$

Esse comportamento também pode ser verificado pelo contador `avaliacoes_arestas`. Como são avaliadas $(n-1)!$ rotas, cada uma com n arestas, a quantidade total de avaliações é:

$$A(n) = n \cdot (n-1)!$$

Logo:

$$\boxed{A(n) = n!}$$

Os resultados obtidos confirmam essa relação em todas as instâncias concluídas.

2.4 Resultados Experimentais e Complexidade Total

Os experimentos foram realizados no seguinte ambiente:

- Processador: Intel Core i5-12450H;
- Memória RAM: 15 GiB;
- Sistema operacional: Ubuntu 24.04.2 LTS;
- Linguagem: Python.

A Tabela 1 apresenta a quantidade de rotas avaliadas, as avaliações de arestas, o valor teórico de $n!$ e os tempos medidos.

Tabela 1 – Resultados dos experimentos com o TSP.

Cidades (n)	**Rotas**	**Avaliações de arestas**	**$n!$ teórico**	**Tempo (s)**
3	2	6	6	0,000008
4	6	24	24	0,000006
5	24	120	120	0,000018
6	120	720	720	0,000091
7	720	5.040	5.040	0,000567
8	5.040	40.320	40.320	0,004326
9	40.320	362.880	362.880	0,035938
10	362.880	3.628.800	3.628.800	0,337044
11	3.628.800	39.916.800	39.916.800	3,518311
12	39.916.800	479.001.600	479.001.600	42,635847
13	479.001.600	6.227.020.800	6.227.020.800	554,932447
14	6.227.020.800	87.178.291.200	87.178.291.200	7.468,000325

Fonte: Dados da pesquisa (2026).

A partir dos resultados, é possível observar que o número de avaliações de arestas corresponde a $n!$, enquanto a quantidade de rotas corresponde a $(n-1)!$.

Como cada rota exige $\Theta(n)$ operações e existem $(n-1)!$ rotas, a complexidade total é:

$$T(n) = (n-1)! \cdot \Theta(n)$$

Simplificando:

$$T(n) = \Theta(n \cdot (n-1)!)$$

Como $n \cdot (n-1)! = n!$, temos:

$$\boxed{T(n) = \Theta(n!)}$$

Portanto, a complexidade obtida experimentalmente está de acordo com a dedução teórica apresentada na unidade.

2.5 Comparação entre a Curva Teórica e a Empírica

Para comparar o comportamento observado com a complexidade teórica, foi gerado um gráfico contendo os tempos medidos e a função:

$$f(n) = c \cdot n!$$

A constante c foi calculada a partir do maior experimento concluído, utilizando o tempo medido para $n = 14$:

$$c = \frac{7468{,}000325}{14!}$$

$$c \approx 8{,}566353185 \times 10^{-8}$$

O gráfico utiliza escala logarítmica no eixo vertical para facilitar a visualização, já que os tempos apresentam diferenças muito grandes entre as menores e as maiores instâncias.

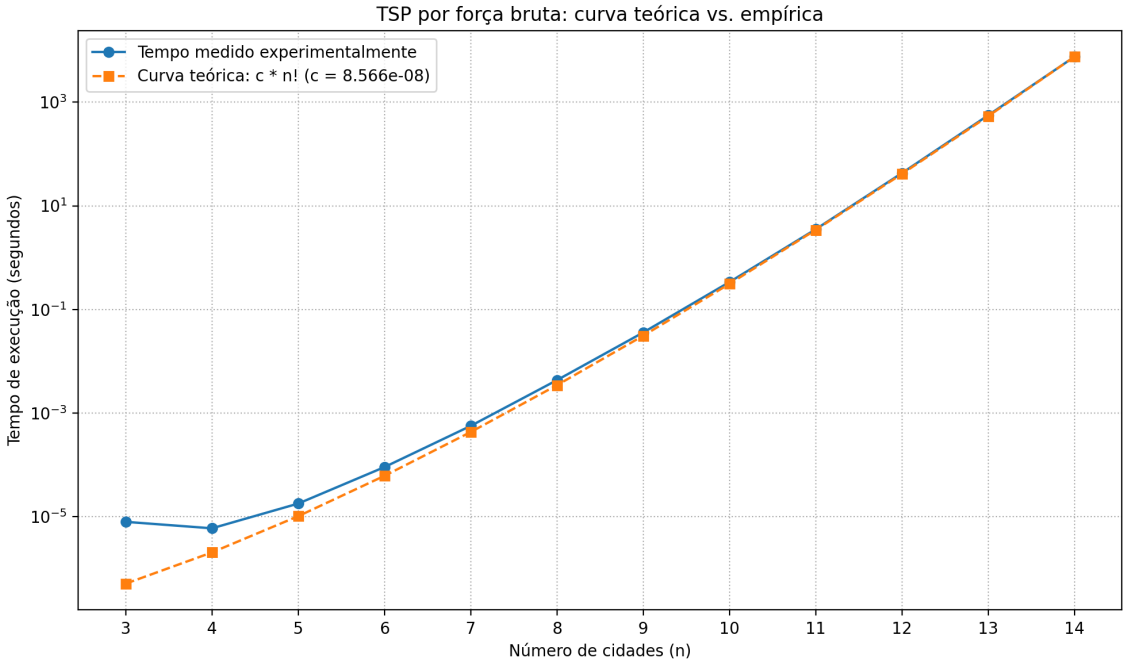


Figura 1 – Comparação entre os tempos medidos e a curva teórica $c \cdot n!$.

Fonte: Dados da pesquisa (2026).

É possível observar que as curvas apresentam comportamentos semelhantes, principalmente nos maiores valores de n. Nos primeiros testes, as diferenças são mais perceptíveis devido à influência dos custos de execução do próprio programa.

Como a constante foi calculada utilizando o resultado de $n = 14$, as curvas coincidem nesse ponto. Essa coincidência é consequência da calibração, e não uma confirmação independente da previsão teórica.

2.6 O Limiar da Intratabilidade

Os resultados mostram que o tempo de execução cresce rapidamente à medida que novas cidades são adicionadas.

Enquanto a execução para 10 cidades levou aproximadamente 0,34 segundo, para 12 cidades foram necessários mais de 42 segundos. Com 13 cidades, o tempo ultrapassou nove minutos, chegando a mais de duas horas para 14 cidades.

A tentativa com 15 cidades ultrapassou 19 horas sem ser concluída.

Esse comportamento está relacionado à complexidade fatorial do algoritmo. Como apresentado na Seção 4.2 da unidade, algoritmos polinomiais, como o Merge Sort, possuem crescimento muito menor. Sua complexidade é:

$$T(n) = \Theta(n \log n)$$

Já algoritmos exponenciais e fatoriais apresentam crescimento muito mais acelerado, podendo se tornar impraticáveis mesmo para entradas relativamente pequenas.

Assim, os experimentos demonstram que o TSP resolvido por força bruta possui limitações práticas importantes, pois o aumento da quantidade de cidades provoca uma explosão no número de possibilidades que precisam ser avaliadas.

2.7 Reflexão sobre P, NP e Heurísticas

A dificuldade do TSP está relacionada à quantidade de possibilidades que precisam ser consideradas para encontrar a rota de menor custo.

Na teoria da complexidade, a classe P reúne problemas que podem ser resolvidos em tempo polinomial, enquanto a classe NP reúne problemas cujas soluções podem ser verificadas em tempo polinomial.

No TSP, verificar uma rota apresentada como solução é uma tarefa relativamente simples: basta conferir se todas as cidades foram visitadas, se o percurso retorna à origem e calcular seu custo. Entretanto, encontrar a rota ótima pode exigir a avaliação de uma quantidade muito grande de possibilidades.

Isso não significa que tenha sido provado que P é diferente de NP. Essa questão continua em aberto na ciência da computação.

Uma alternativa à força bruta seria utilizar uma heurística gananciosa (Greedy). Nesse caso, o algoritmo poderia escolher, a cada etapa, a cidade não visitada com menor custo de deslocamento em relação à cidade atual.

Essa estratégia pode encontrar uma rota rapidamente, mas não garante que ela seja a melhor possível. Uma escolha que parece vantajosa em determinada etapa pode resultar em um custo total maior ao final do percurso.

Por isso, heurísticas são utilizadas para encontrar boas soluções em menos tempo, sem necessariamente garantir a solução ótima. As metaheurísticas, por sua vez, utilizam estratégias de busca mais elaboradas para explorar diferentes possibilidades e melhorar os resultados encontrados.

Dessa forma, embora Greedy possa ser uma alternativa mais rápida, ele não substitui a força bruta quando o objetivo é garantir a rota de menor custo.

3 CONCLUSÃO

A implementação do TSP permitiu verificar, na prática, o comportamento de um algoritmo de força bruta e sua relação com a complexidade teórica.

A fixação da cidade inicial reduziu a quantidade de rotas avaliadas para $(n-1)!$. Além disso, foi possível confirmar que cada rota exige n avaliações de arestas, resultando na complexidade total $\Theta(n!)$.

Os experimentos também demonstraram o impacto desse crescimento no tempo de execução. Para 14 cidades, foram necessárias mais de duas horas, enquanto a tentativa com 15 cidades ultrapassou 19 horas sem ser concluída.

Assim, os resultados mostram que, apesar de garantir a solução ótima, a força bruta possui limitações para instâncias maiores. A comparação com a curva teórica e a discussão sobre P, NP e heurísticas permitiram compreender melhor a relação entre a complexidade de um algoritmo e sua viabilidade prática.

Referências

Universidade Vila Velha. *Projeto e Análise de Algoritmos: Capítulo 4*. Vila Velha, ES, 2026. Material didático da disciplina.